

Image Classification with Ensemble Stacking Techniques

Yiming(Eric) Cao
92138098
Shauna(Shengnan) Song
94719218

Group 29

May 8, 2023

1 Goal of Report

The primary goal of this project is to enhance image classification accuracy by employing ensemble stacking techniques, which effectively combine the strengths of multiple neural network models, such as Convolutional Neural Networks (CNNs). By minimizing loss and reducing overfitting, we aim to improve the overall stability and robustness of the image classification system. Furthermore, we strive to demonstrate that the ensemble stacking approach yields better performance in comparison to individual models used separately. In doing so, we hope to develop a more comprehensive understanding of how ensemble techniques can be applied to image classification tasks.

2 Previous Work Done

Ensemble learning techniques have garnered significant attention in the field of machine learning and have been successfully applied in various domains, including image classification tasks [1]. These techniques essentially combine the output of multiple models, or base learners, to improve the overall prediction accuracy and reliability of the system. By leveraging the strengths of each model and compensating for their individual weaknesses, ensemble learning has been proven to yield better performance than individual models in many cases [2].

Two of the most popular ensemble learning techniques are boosting and bagging. Boosting is an iterative approach that focuses on improving the performance of weak classifiers by combining their outputs in a weighted manner. Each weak classifier's weight is determined by its accuracy, with more accurate classifiers receiving higher weights [3]. Bagging, on the other hand, is a technique that aims to reduce variance by training multiple base learners independently and averaging their predictions. This approach helps to mitigate the effect of overfitting and improve the overall stability of the model [4].

Convolutional Neural Networks (CNNs) have been particularly successful in the domain of image classification due to their ability to automatically learn spatial hierarchies of features directly from the input data [5]. Over the years, various CNN architectures have been proposed, each with their unique strengths and contributions to the field of computer vision. Some of these influential architectures include LeNet [6], AlexNet [7], ResNet [8], and VGG [9].

LeNet [5], was one of the first successful applications of CNNs for image recognition. This pioneering work laid the foundation for the development of more advanced CNN architectures by demonstrating the effectiveness of convolutions and pooling layers for capturing local spatial information in images.

AlexNet [10], built upon the foundation laid by LeNet and significantly increased the depth and size of the network. AlexNet's success in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 marked a turning point in the field of computer vision, as it surpassed traditional handcrafted feature-based methods by a significant margin. This achievement sparked renewed interest in the development of deep learning methods for image classification.

ResNet[11], tackled the issue of vanishing gradients in very deep networks by incorporating skip connections, or residual connections, which allow the network to learn residual functions with reference to the layer inputs. This innovation enabled the training of much deeper networks, with ResNet achieving state-of-the-art performance on the ILSVRC and COCO object detection and segmentation tasks [11].

VGG[12], is another deep CNN architecture that achieved remarkable performance on the ILSVRC. The main contribution of VGG was its demonstration that the depth of the network is a critical factor in achieving good performance. VGG12 consists of 12 weight layers, including 9 convolutional layers and 3 fully connected layers, and makes use of small 3x3 convolutional filters throughout the network.

The idea of combining multiple CNN models to improve performance has been explored in several studies. For instance, the work of Szegedy et al. [13] proposed an architecture called Inception that combined multiple convolutional filter sizes in parallel at each layer to capture spatial information at various scales. Similarly, Huang et al. [14] introduced Densely Connected Convolutional Networks (DenseNets), which connect each layer to every other layer in a feed-forward fashion to facilitate the flow of information and gradients between layers.

Ensemble stacking is a popular technique that aims to build a more robust model by stacking multiple base models and training a meta-model on their predictions [15]. The basic idea is to use the base models to generate predictions, which are then used

3 Strategy

To achieve our goal of enhancing image classification accuracy, we will employ ensemble stacking techniques to combine the strengths of multiple Convolutional Neural Network (CNN) models, such as LeNet, AlexNet, ResNet18, and VGG12. Each of these CNN models has unique characteristics and strengths, which contribute to capturing various aspects of the image data. By combining their predictions, we can leverage their collective intelligence to create a more robust and accurate image classification system.

Our strategy involves the following steps:

1. Data preparation: We will use the CIFAR-10 dataset, a widely-used benchmark for image classification tasks, consisting of 60,000 color images of size 32x32 pixels, distributed evenly across 10 distinct classes. The dataset will be split into training and testing sets, ensuring an equal representation of each class in both sets.
2. Model selection: We will choose a set of diverse and complementary CNN models that have demonstrated success in image classification tasks. These models will serve as our base models in the ensemble stacking approach. The selected models include LeNet, AlexNet, ResNet18, and VGG12, each with their unique architectural features and strengths.
3. Meta-model design and training: We will design another neural network, which will serve as our meta-model. The meta-model's input will be the stacked predictions from the base models, and its output will be the final predicted class probabilities. The meta-model will be trained using the same training techniques as the base models but with a focus on minimizing the classification error based on the aggregated predictions.
4. Performance evaluation: We will assess the overall performance of our ensemble stacking approach by comparing the classification accuracy of the combined model against the individual base models. This evaluation will provide insights into the effectiveness of the ensemble stacking technique in improving image classification accuracy.

By implementing this strategy, we expect to create a more robust image classification system that benefits from the collective strengths of multiple CNN models. The ensemble stacking approach is anticipated to result in reduced overfitting and improved overall performance compared to using individual models separately.

4 What Methods Worked

4.1 Overview

In this project, our goal is to complete an image classification task and improve image classification accuracy. We follow these steps:

- Load the data and split it into train, validation, and test datasets.
- Perform data augmentation on the loaded dataset.
- Train multiple types of CNNs as base learners and assess their performance.
- Implement an ensemble method as a meta-learner by stacking a variety of pre-trained CNNs.
- Evaluate the performance of the meta-learner and compare it with the performance of the base learners.

Regarding the general settings, all of the code for this project was developed using the Python programming language, relying on widely-used libraries such as NumPy, PyTorch, and others. To facilitate experimentation and collaboration, the code was organized in the form of a Jupyter notebook, and the computations were performed using GPU-accelerated environments provided by Google Colab.

In terms of the training strategy, we carefully tuned the hyperparameters of the models and experimented with various ensemble stacking configurations to optimize the overall classification accuracy. This iterative process allowed us to identify the methods that worked well, as well as those that did not, thereby providing valuable insights into the effectiveness of ensemble stacking techniques in the context of image classification tasks.

4.2 Load the data set: CIFAR-10

The CIFAR[16] datasets have become popular benchmarks for image recognition tasks due to their diverse set of images and relatively small size. They consist of color images that are 32x32 pixels in size and are labeled into 10 different classes. The datasets were collected by researchers Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton, and are widely used for evaluating the performance of image recognition models. The small size of the datasets and the relatively low computational requirements for processing them make them a popular choice for developing and testing image recognition algorithms.

In this project, we use the `datasets` method in the `torchvision` class to obtain the **CIFAR10** train and test set, and use `random split` method in `torch.utils.data` to split the validation set out of the train set, the corresponding **Python** code is in Appendix A.

4.3 Data preprocessing: Data Augmentation

Data augmentation is a widely used technique in computer vision tasks, aimed at increasing the size and diversity of a dataset by applying various image transformations. In this project, we employed several data augmentation methods to enhance the training data and improve the model's generalization and robustness. The following data augmentation techniques were applied:

- **Horizontal Flipping:** This method involves flipping the image horizontally, effectively creating a mirror image. This increases the dataset size and helps the model learn to recognize objects in different orientations.
- **Color Jittering:** This method randomly alters the image's brightness, contrast, saturation, and hue, simulating various lighting conditions and teaching the model to recognize objects with different color representations.
- **Rotation:** This involves rotating the image by a certain angle to help the model recognize objects from different perspectives.

In order to implement data augmentation in the code, we use the `transforms` method in the `torchvision` class, the corresponding **Python** code is in Appendix B.

Data augmentation is a beneficial technique for improving models' performance, helping improve the model's accuracy by making it more robust and generalizing to variations in the input data. It is important to note that the augmented data should be kept within the feasible range of the dataset. For instance, vertical flipping is inappropriate for our task. Besides, we should not introduce too much augmentation into the images, which can cause the model to overfit the training data.

4.4 CNN: LeNet

LeNet [5] was introduced in 1998 as one of the earliest successful CNN architectures, designed specifically for recognizing handwritten digits in images. An implementation of LeNet using **Python** and **PyTorch** is in Appendix C

As an early successful CNN in the field of computer vision and deep learning, LeNet is known for its straightforward architecture and high accuracy in recognizing digits. We plan to use LeNet as our

initial CNN for processing the CIFAR-10 dataset. The pseudocode for training is in Appendix D, and the algorithm in Appendix E presents the pseudocode for evaluating the model on the validation set (during training) and the test set (after training).

In this project, use **Adam** optimizer that imports from **torch.optim** to train the LeNet described above six times with learning rates 0.001 and 0.0001(each three times) by 100 epochs. The, result is below:

Table 1: Evaluate LeNet with variety of learning rates on test set

	learning rate	Accuracy (%)
LeNet1	0.001	66.93
LeNet2	0.001	67.55
LeNet3	0.001	68.02
LeNet4	0.0001	65.76
LeNet5	0.0001	65.85
LeNet6	0.0001	65.97

Our selection strategy of learning rate is based on the loss and accuracy curves. When a curve graph behaves as follows, we judge that using this learning rate can make the model converge to optimal classification accuracy.

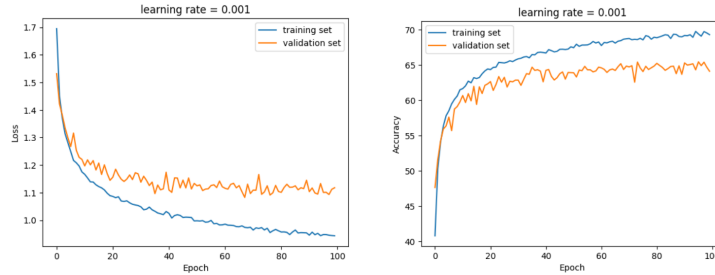


Figure 1: loss(left) and accuracy(right) curves

In the following CNNs, we will use the same training strategy.

4.5 CNN: AlexNet

To achieve faster convergence with AlexNet, we employed a model pre-trained on ImageNet [17] and fine-tuned it on CIFAR [16]. During fine-tuning, the parameters of the convolutional layers remained fixed, with only the outermost fully connected layer used for classification being trained. We utilized the **Adam** optimizer, imported from **torch.optim**, to train the aforementioned AlexNet model. The training was conducted six times, alternating learning rates of 0.001 and 0.0001 (every three trials), for a total of 100 epochs each. The results are presented below:

Table 2: Evaluate AlexNet with variety of learning rates on test set

	learning rate	Accuracy (%)
AlexNet1	0.001	83.63
AlexNet2	0.001	83.26
AlexNet3	0.001	82.95
AlexNet4	0.0001	85.58
AlexNet5	0.0001	85.58
AlexNet6	0.0001	84.96

AlexNet has a higher learning capacity than LeNet due to its deeper architecture with more convolutional layers, which results in improved performance. Considering the size and complexity of

the cifar10 dataset, this network has the most optimal performance among all the base learners in this project, due to AlexNet’s appropriately complex structure.

4.6 CNN: VGG

For the VGG model, we adopted the same training strategy as with AlexNet, using a model pre-trained on ImageNet [17] and fine-tuning it on CIFAR [16]. During fine-tuning, the parameters of the convolutional layers remained fixed, while only the outermost fully connected layer used for classification was trained. We utilized the **Adam** optimizer, imported from **torch.optim**, to train the VGG model. The training was conducted six times, alternating learning rates of 0.001 and 0.0001 (every three trials), for a total of 10 epochs each. The results are presented below:

Table 3: Evaluate VGG12 with variety of learning rates on test set

	learning rate	Accuracy (%)
VGGNet1	0.001	82.89
VGGNet2	0.001	83.56
VGGNet3	0.001	83.06
VGGNet4	0.0001	84.17
VGGNet5	0.0001	84.06
VGGNet6	0.0001	84.04

Although one might expect that VGG, with its deeper structure, would outperform other architectures, experimental results indicate the opposite. This can be attributed to the vanishing gradient problem, as illustrated in the graph below, where both training and testing losses increase after a certain point. To address this issue, early stopping is employed (stopping at the 10th epoch), but this approach may result in an underfit network.



Figure 2: Gradient vanishing on VGG12 for training with 100 epochs

4.7 CNN: ResNet

For the ResNet model, we adopted the same training strategy as before and utilized the **Adam** optimizer, imported from **torch.optim**, to train the model. The training was conducted six times, with learning rates of 0.0001, for 10 epochs. The results are presented below:

Table 4: Evaluate ResNet on test set

	learning rate	Accuracy (%)
ResNet1	0.0001	83.59
ResNet2	0.0001	84.10
ResNet3	0.0001	84.21
ResNet4	0.0001	83.86
ResNet5	0.0001	84.29
ResNet6	0.0001	84.15

Similar to VGG, ResNet also has a deep structure. However, it includes residual connections between the ends of each convolutional block. These connections help overcome the vanishing gradient problem observed in VGG, allowing ResNet to learn a better-performing model. However, ResNet tends to overfit due to its deep structure.(shown in the graph below). To address this issue, early stopping is employed (stopping at the 10th epoch)

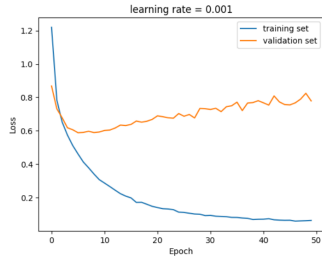


Figure 3: Overfitting on Resnet18 for training with 50 epochs

4.8 Ensemble meta-learning: Stacking

We opted for ensemble stacking because it has been demonstrated to enhance the accuracy of image classification tasks in comparison to individual models [18]. The integration of multiple CNNs assists in reducing both bias and variance, leading to superior classification performance. Our results show that the ensemble stacking technique surpasses the performance of individual CNN models on the CIFAR-10 dataset.

We accomplished ensemble stacking by fusing the predictions of several models to boost the overall performance of the ensemble. The pseudocode example illustrating how to stack three models is shown in Appendix F.

For training the meta model, we utilized the **Adam** optimizer, imported from **torch.optim**. we used learning rates of 0.001 for a total of 10 epochs. The results are presented below:

Table 5: Ensemble stacking multiple identical models

	highest base learner accuracy (%)	Meta learner accuracy (%)	improvement (%)
LeNet(stack 3)	68.02	70.90	2.88
LeNet(stack 6)	68.02	70.98	2.96
AlexNet(stack 3)	85.58	87.88	2.30
AlexNet(stack 6)	85.58	87.89	2.31
VGG12(stack 3)	84.17	86.19	2.02
VGG12(stack 6)	84.17	86.40	2.23
ResNet18(stack 3)	84.29	87.09	2.80
ResNet18(stack 6)	84.29	87.37	3.08

By stacking three and six identical models of each type of CNN, we observed that ensemble learning improved classification accuracy by 2% – 3% for each type of CNN. When comparing the performance of stacking three models to stacking six models, the latter consistently exhibited marginally better results. However, it is important to note that stacking six models required more computational time during training. In addition, we also conducted tests on stacking three different models, and here are the results:

The performance achieved by stacking various CNNs is remarkably impressive, with the LeNet, AlexNet, and VGG12 combination yielding the most optimal result in this project. Discussion on ensemble learning will be covered in the following two sections.

Table 6: Ensemble stacking multiple different models

	Meta learner accuracy (%)
AlexNet + ResNet18 + VGG12	88.51
LeNet + AlexNet + VGG12	88.67
LeNet + ResNet18 + VGG12	86.77
LeNet + AlexNet + ResNet18	87.82

5 What did not work

Throughout our efforts to achieve the results above, we faced several challenges and limitations, which ultimately impacted our ability to optimize the ensemble model’s performance. These challenges include the inability to measure bias and variance directly, the blind selection of models for stacking, and the absence of transformer models in our exploration.

One of the primary challenges we faced was the difficulty in quantifying the bias and variance of each trained model, regardless of whether they were of the same or different types. Bias typically refers to the deficiency in a model’s learning capability, which causes a discrepancy between the predicted and actual values. High bias usually indicates underfitting, while high variance, which represents the extent to which a model is sensitive to variations in data, often results in overfitting. Compared to other traditional ensemble methods, bagging, reduced variance; boosting, reduced bias. Ensemble stacking aims to achieve a balance between bias and variance to obtain an optimal fit between the model and the data. However, these theoretical concepts cannot be directly measured on the models themselves, hindering our ability to identify the exact contributions of individual models to the overall ensemble’s performance.

As a result of this challenge, we were unable to selectively stack models based on their bias and variance characteristics. Instead, we had to rely on a more blind or exhaustive approach to test and select which models to include in the ensemble. This approach can be computationally expensive and inefficient, as it requires testing multiple combinations of models to find the optimal stacking configuration. Similar to the experiment of stacking diffnet CNNs at the end of this project, these experimental results are non-interpretable due to this reason.

Another limitation we encountered was the exclusion of transformer models from our investigation. Transformer models have been shown to be highly effective in various machine learning tasks, including image classification. By not incorporating transformers into our study, we may have missed out on valuable insights and potential performance improvements that these models could have offered to our ensemble.

Moreover, expanding our analysis to include other model architectures, such as transformers, could have provided a more comprehensive understanding of the benefits and limitations of various stacking configurations. The inclusion of a broader range of models would also increase the diversity of our ensemble, potentially leading to further improvements in classification accuracy.

In summary, our work faced challenges in measuring and utilizing the bias and variance of individual models, selecting models for stacking based on their theoretical properties, and incorporating a diverse range of model architectures, such as transformers. These limitations impacted our ability to optimize the ensemble model’s performance fully and may have left room for further improvements. By addressing these challenges in future work, we could potentially enhance our understanding of ensemble learning techniques and their effectiveness in image classification tasks.

6 Conclusions and Future Work

In this project, we have successfully demonstrated the potential of the ensemble stacking technique in improving image classification accuracy by leveraging the strengths of multiple Convolutional Neural Network (CNN) models. Ensemble learning aims to achieve a balance between bias and variance to achieve a perfect fit between the model and the data. Ensemble stacking achieves this balance by averaging each model with equal weights, thereby reducing the bias and variance of each base learner to the lowest possible level. Our results indicate that the ensemble stacking technique outperforms

individual models, showcasing the benefits of combining diverse and complementary models to create a more robust and accurate classification system.

Future work in this area could involve several avenues of exploration. Firstly, evaluating the improvement of ensemble learning using Vision Transformers (ViT) could be a promising direction. ViT has recently gained attention for its excellent performance in image classification tasks, outperforming traditional CNNs in certain scenarios. Incorporating ViT models into the ensemble stacking approach could potentially lead to further improvements in classification accuracy and provide insights into the benefits and limitations of combining transformers with CNNs.

Another direction for future research could be the exploration of model soups, which aim to improve accuracy by averaging the weights of multiple models' parameters without increasing inference time. Model soups can offer the benefits of ensemble learning without the added complexity and computational overhead typically associated with stacking or other ensemble techniques. Investigating the effectiveness of model soups in image classification tasks and comparing their performance with ensemble stacking could provide valuable insights into the trade-offs between accuracy, complexity, and computational efficiency.

In conclusion, our work has demonstrated the potential of ensemble stacking techniques in improving image classification accuracy by combining the strengths of multiple CNN models and striking a balance between bias and variance. Our findings contribute to the growing body of research on ensemble learning techniques and their application in image classification tasks. Further exploration of alternative ensemble approaches, such as incorporating Vision Transformers and investigating model soups, can provide additional insights into the best practices for leveraging the collective intelligence of diverse models to enhance image classification performance.

References

- [1] Y. Yang, N. Chen, and S. Jiang, “Collaborative strategy for visual object tracking,” *Multimedia Tools and Applications*, vol. 77, no. 6, p. 7283–7303, 2018.
- [2] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. CRC Press, 2012.
- [3] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [4] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [6] M. Kayed, A. Anter, and H. Mohamed, “Classification of garments from fashion mnist dataset using cnn lenet-5 architecture,” in *2020 International Conference on Innovative Trends in Communication and Computer Engineering (ITCE)*, pp. 238–243, 2020.
- [7] S. Arya and R. Singh, “A comparative study of cnn and alexnet for detection of disease in potato and mango leaf,” in *2019 International Conference on Issues and Challenges in Intelligent Computing Techniques (ICICT)*, vol. 1, pp. 1–6, 2019.
- [8] S. Targ, D. Almeida, and K. Lyman, “Resnet in resnet: Generalizing residual architectures,” *arXiv preprint arXiv:1603.08029*, 2016.
- [9] H. Qassim, A. Verma, and D. Feinzimer, “Compressed residual-vgg16 cnn model for big data places image recognition,” in *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 169–175, 2018.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Advances in Neural Information Processing Systems*, pp. 1097–1105, 2012.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *arXiv preprint arXiv:1512.03385*, 2016.
- [12] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [13] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1–9, 2015.
- [14] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” *arXiv preprint arXiv:1704.06904*, 2017.
- [15] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [16] A. Krizhevsky, G. Hinton, *et al.*, “Learning multiple layers of features from tiny images,” 2009.
- [17] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,”
- [18] F. Wang, M. Jiang, C. Qian, S. Yang, C. Li, H. Zhang, X. Wang, and X. Tang, “Residual attention network for image classification,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 7, pp. 1619–1632, 2018.

Appendix A Load CIFAR10 dataset

```
def load_CIFAR10(train_transforms, test_transforms):
    dataset = datasets.CIFAR10(root='./data',
                                train=True,
                                download=True,
                                transform=train_transforms)
    test_dataset = datasets.CIFAR10(root='./data',
                                     train=False,
                                     download=True,
                                     transform=test_transforms)

    train_dataset, val_dataset = randosm_split(dataset, [0.8, 0.2])

    train_loader = torch.utils.data.DataLoader(train_dataset,
                                                batch_size=100,
                                                shuffle=True,
                                                num_workers=12)
    val_loader = torch.utils.data.DataLoader(val_dataset,
                                              batch_size=100,
                                              shuffle=True,
                                              num_workers=12)
    test_loader = torch.utils.data.DataLoader(test_dataset,
                                              batch_size=100,
                                              shuffle=False,
                                              num_workers=12)

    return train_loader, val_loader, test_loader
```

Appendix B Data augmentation

```
def transform(img_size):
    train_transforms = transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(5),
        transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),])
    # No data augmentation on the test set.
    test_transforms = transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),])
    return train_transforms, test_transforms
```

Appendix C LeNet

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(nn.Conv2d(3, 16, 5), nn.MaxPool2d(2, 2))
        self.conv2 = nn.Sequential(nn.Conv2d(16, 32, 5), nn.MaxPool2d(2, 2))
        self.fc1 = nn.Linear(32*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = x.view(x.size(0), -1) # flatten
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)
        return x
```

Appendix D Training algorithm

Algorithm 1 Training

Input: Network, train set, validation set

Output: train accuracy, train loss, validation accuracy, validation loss

```
train accuracy = []
train loss = []
validation accuracy = []
validation loss = []
for i = 1, 2,..., epoch do
    corrects = 0
    losses = 0
    data = iter(train set)
    for j = 1, 2,..., len(train set) do
        image batches, labels = next(data)
        prediction = Network(image batches)
        loss = CrossEntropy(prediction, labels)

        clear gradients from the last step
        compute loss gradient by backpropagation
        update network parameters based on gradient

        corrects = corrects + nonzeros(prediction == labels)
        losses = losses + loss
    end for
    train accuracy.append(corrects / len(train set))
    train loss.append(losses / len(train set))
    val acc, val loss = evaluate(Network, validation set)
    validation accuracy.append(val acc)
    validation loss.append(val loss)
end forreturn train accuracy, train loss, validation accuracy, validation loss
```

Appendix E Evaluate algorithm

Algorithm 2 Evaluate

Input: Network, data set

Output: accuracy, losses

```
1: corrects = 0
2: losses = 0
3: data = iter(data set)
4: for j = 1, 2,..., len(data set) do
5:     image batches, labels = next(data)
6:     prediction = Network(image batches)
7:     loss = CrossEntropy(prediction, labels)
8:
9:     corrects = corrects + nonzeros(prediction == labels)
10:    losses = losses + loss
11: end for
12: accuracy = corrects / len(data set)
13: losses = losses / len(data set)
```

Appendix F Ensemble stacking with three base learners

```
class MetaNet3(nn.Module):
    def __init__(self, modelA, modelB, modelC):
        super().__init__()
        self.base1 = modelA
        self.base2 = modelB
        self.base3 = modelC
        self.meta = nn.Linear(10 * 3, 10)

    def forward(self, x):
        x1 = self.base1(x)
        x2 = self.base2(x)
        x3 = self.base3(x)
        x = torch.cat((x1, x2, x3), dim=1)
        return self.meta(x)
```